

Ecosistema de Automatización IA Autónomo para Negocios

Pipeline de generación y distribución de contenido con validación humana

Caso de uso	Festín Café Bistró — La Molina, Lima
Autor	Guillermo Manosalva
Curso	Diseño de Agentes y Automatización
Fecha	26 de septiembre de 2026
Stack	Airtable · Make · Google Gemini · Gmail

Los cinco criterios de evaluación

Criterio	Peso	Dónde se resuelve	Pág.
1 · Mapa de arquitectura	20%	Diagrama del ecosistema e inventario de módulos	3-5
2 · Estructuras de datos	20%	Esquema relacional y contratos JSON de transferencia	6-9
3 · Optimización de costos	20%	Matriz de decisión por tarea y ahorro calculado	10-11
4 · Seguridad y resiliencia	20%	Error handlers, HITL y minimización de datos	12-16
5 · Dashboard de control	20%	Panel de KPIs y enlaces públicos	17

Qué demuestra este documento

Un sistema que ya corrió de punta a punta: genera contenido consultando una base de conocimiento propia, se detiene a esperar la aprobación de una persona, publica solo lo aprobado y registra cada ejecución —incluidos los fallos— en una tabla de auditoría con su propio panel de control. Las capturas provienen de once ejecuciones reales realizadas el 26 de septiembre de 2026.

Resumen del sistema

Festín es un café bistró de La Molina que publica en Instagram y Facebook. El problema no es la falta de ideas, sino el cuello de botella entre tenerlas y publicarlas: alguien tiene que redactar, revisar y subir cada pieza, y en la práctica eso se posterga. El sistema automatiza la redacción y la distribución, pero deja intacta la decisión de publicar —que sigue siendo humana.

11	6	27,3%	3
ejecuciones registradas	pruebas de estrés	tasa de error medida	tablas vinculadas

Cómo funciona, en una frase por etapa

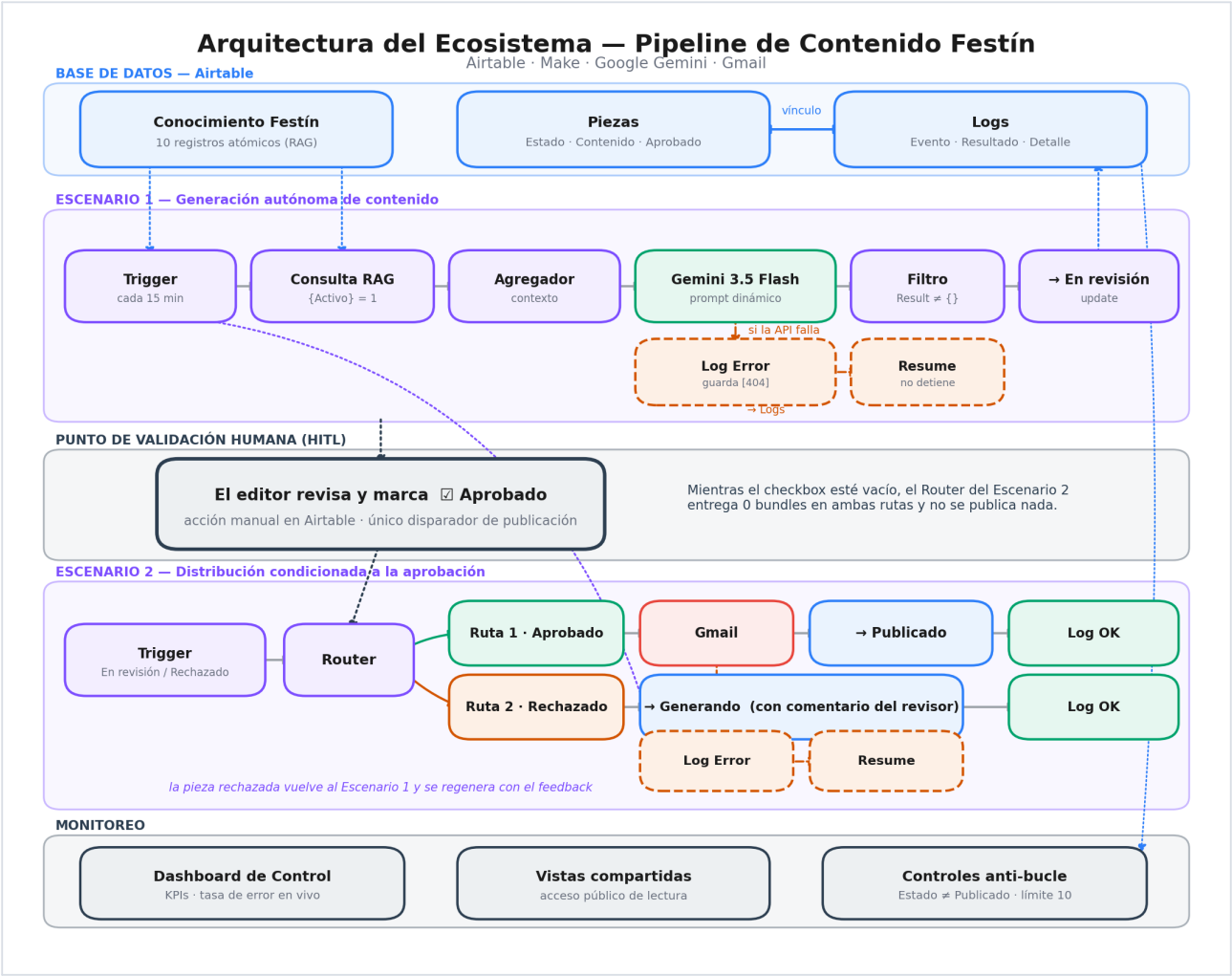
- Detecta. Cada 15 minutos Make busca en Airtable las piezas marcadas como «Generando» que tengan una idea semilla escrita.
- Se informa. Antes de redactar, consulta una base de conocimiento con datos reales del local: carta, precios, horarios, dirección, tono de marca.
- Redacta. Gemini recibe ese contexto junto con la idea semilla y el canal de destino, y devuelve el post listo.
- Se detiene. La pieza pasa a «En revisión» y ahí se queda. Ningún automatismo la mueve de ahí.
- Espera. Un editor lee el texto y marca —o no— la casilla «Aprobado» en Airtable.
- Publica o corrige. Solo si la casilla está marcada, el segundo escenario notifica por Gmail y marca la pieza como publicada. Si en cambio recibe un comentario de rechazo, la devuelve al inicio con ese feedback incorporado.
- Registra. Cada paso —exitoso o fallido— escribe una fila en la tabla de Logs, que alimenta el dashboard.

La decisión de diseño que sostiene todo

El sistema está partido en dos escenarios independientes en lugar de uno solo. No es una complicación gratuita: un flujo continuo no puede esperar a una persona sin quedarse colgado consumiendo operaciones. Al separarlos, el primero termina su trabajo y se apaga; el segundo corre en paralelo y solo encuentra trabajo que hacer cuando alguien ya aprobó algo. El estado en Airtable es lo que comunica a ambos.

Criterio 1 · Mapa de arquitectura

Diagrama completo del ecosistema: disparadores, enrutadores, APIs, nodos de IA y destino de los datos en cada rama.



Inventario de módulos

Los once módulos del sistema, con su nombre real en Make. La consigna exige nodos nombrados de forma que se entienda qué hace cada uno sin abrirlo.

Escenario 1 — Generación

#	Módulo	Tipo	Función
1	Trigger - Piezas en Generando	Airtable · Search	Busca piezas listas para redactar
2	Consulta RAG - KB Festín	Airtable · Search	Trae los datos activos del local
3	Contexto RAG agregado	Text aggregator	Une los registros en un solo bloque
4	IA - Redacción Gemini	Gemini · Generate	Redacta el post con el contexto
5	Estado → En revisión	Airtable · Update	Guarda el texto y cambia el estado
6	Resume	Flow control	Evita que un fallo detenga el escenario
8	Log Error - Gemini	Airtable · Create	Registra el fallo de la API
9	Log OK - Generación	Airtable · Create	Registra la generación exitosa

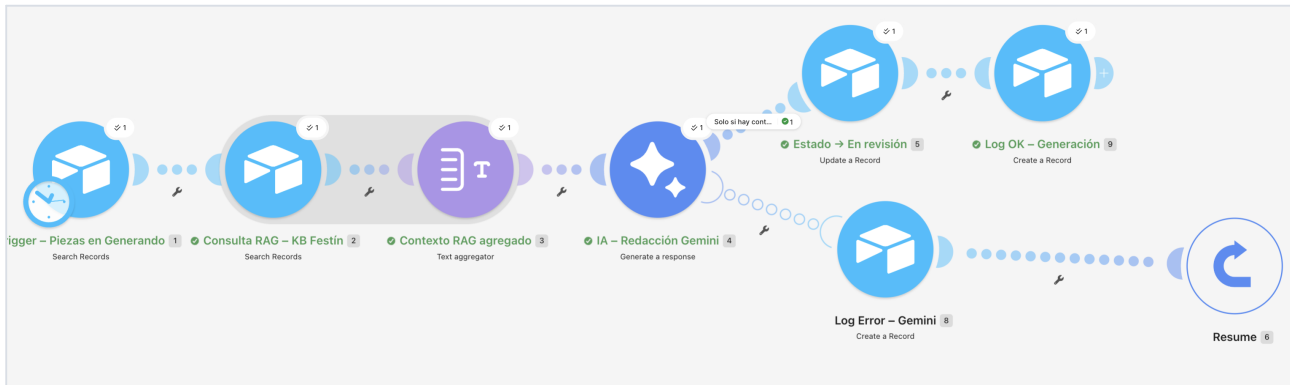
Escenario 2 — Distribución

#	Módulo	Tipo	Función
1	Buscar piezas en revisión	Airtable · Search	Trae piezas en revisión o rechazadas
2	Router	Flow control	Separa aprobadas de rechazadas
3	Gmail - Notificación	Gmail · Send	Avisa que la pieza se publicó
4	Airtable - Devolver	Airtable · Update	Regresa la pieza a Generando
6	Airtable - Publicar	Airtable · Update	Marca la pieza como Publicado
7	Resume	Flow control	Absorbe fallos de Gmail
8	Log Error - Gmail	Airtable · Create	Registra el fallo de envío
9	Log OK - Publicado	Airtable · Create	Registra la publicación
10	Log OK - Rechazo	Airtable · Create	Registra la devolución a corrección

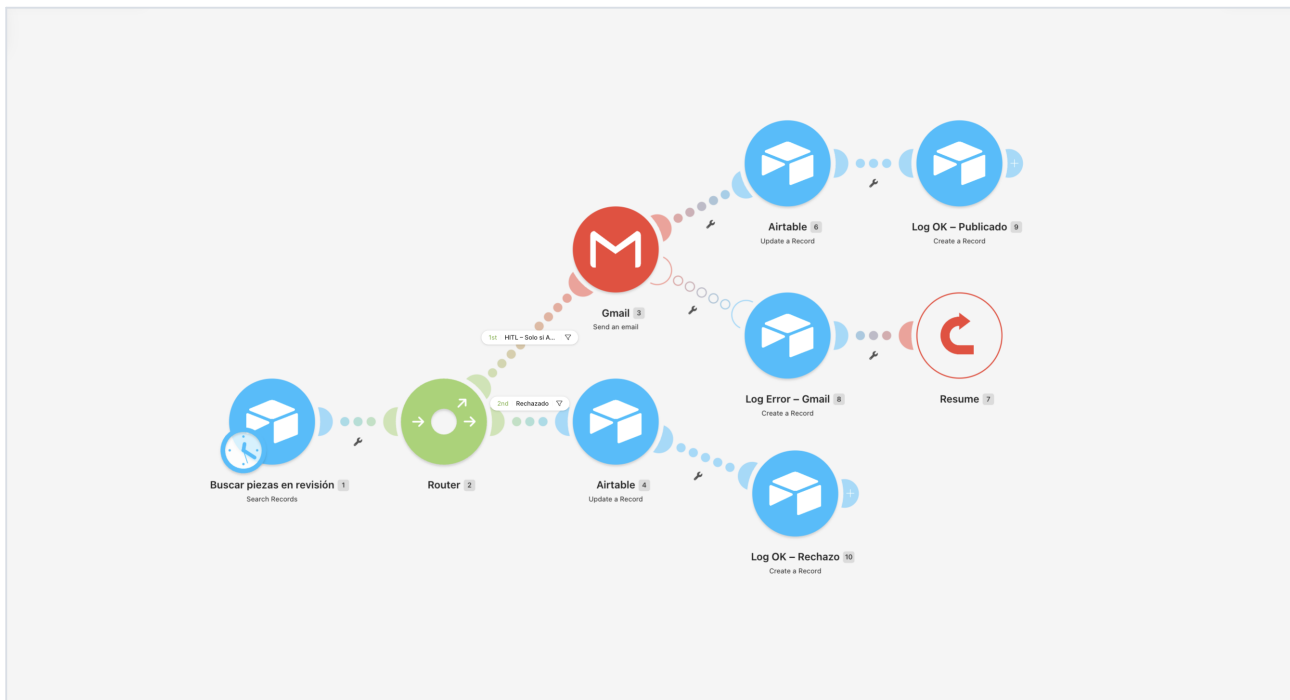
Sin datos hardcodedos

Ningún módulo contiene texto fijo del negocio. La dirección, el WhatsApp, los precios y el tono salen de la tabla de conocimiento; la idea y el canal salen de la fila que se está procesando. Cambiar el horario del local no exige tocar Make: se edita un registro en Airtable y la siguiente ejecución ya lo usa.

Los dos escenarios en producción



Escenario 1. La rama inferior es la ruta de error: se activa solo si Gemini falla.



Escenario 2. El Router abre dos caminos excluyentes y cada uno cierra con su propio registro en Logs.

Criterio 2 · Estructuras de datos

El sistema usa dos bases de Airtable. La de conocimiento alimenta al modelo; la de comando guarda el trabajo y su historial, con dos tablas vinculadas entre sí.

Base A — Conocimiento Festín

Memoria de largo plazo. Cada fila es un dato atómico y verificable del negocio; el escenario lee solo las que están activas.

Campo	Tipo	Para qué sirve
Tema	Texto	Etiqueta del dato (ej. «Horario», «Dirección»)
Categoría	Selección única	Agrupar los datos por familia
Contenido	Texto largo	El hecho en sí (ej. «5 pm a 10 pm»)
Activo	Casilla	Interruptor: si está apagado, el dato no entra al prompt

Base B — Centro de Comando · tabla Piezas

El estado de cada pieza de contenido a lo largo de su vida.

Campo	Tipo	Para qué sirve
Idea Semilla	Texto	Entrada humana: de qué trata el post
Canal	Selección única	Instagram o Facebook — cambia el formato del texto
Estado	Selección única	Generando → En revisión → Publicado / Rechazado
Contenido Generado	Texto largo	Salida del modelo
Aprobado	Casilla	El punto de control humano
Comentario Revisión	Texto largo	Feedback que se reinyecta al prompt al regenerar
Última Ejecución	Fecha y hora	Cuándo la tocó el sistema por última vez
Logs	Vínculo → Logs	Todas las ejecuciones asociadas a esta pieza

Base B — tabla Logs

Bitácora de auditoría. Es la tabla que hace medible al sistema.

Campo	Tipo	Para qué sirve
Evento	Texto	Qué ocurrió, en lenguaje llano
Pieza	Vínculo → Piezas	Sobre qué contenido ocurrió
Escenario	Selección única	Esc 1 - Generación · Esc 2 - Distribución
Resultado	Selección única	OK · Error — de aquí sale la tasa de error
Módulo	Texto	Dónde exactamente ocurrió
Detalle	Texto largo	Mensaje literal devuelto por la API que falló
Timestamp	Hora de creación	Sello temporal automático

Por qué las dos tablas están vinculadas

La consigna pide relaciones entre tablas «para evitar datos aislados». Un log suelto que dice «falló Gemini» no sirve para nada: hay que saber sobre qué pieza falló. El campo Pieza es un vínculo real de Airtable, no un texto copiado, y genera el campo inverso Logs en la tabla de contenido. Desde cualquier pieza se ve su historial completo, y desde cualquier error se llega en un clic al contenido que lo provocó.

Contratos JSON entre integraciones

Cuatro transferencias componen el ciclo. Estos son los objetos que viajan en cada una.

1 · Airtable → Make (salida del trigger)

```
{
  "id": "rec9JIOXG5jp3VhNV",
  "Idea Semilla": "Café de especialidad por las mañanas",
  "Canal": "Instagram",
  "Estado": "En revisión",
  "Aprobado": true,
  "Comentario Revisión": "",
  "Total number of bundles": 6
}
```

El campo Aprobado solo aparece cuando la casilla está marcada. Airtable omite los booleanos falsos, y esa particularidad es la que se aprovecha en la condición del Router.

2 · Make → Gemini (cuerpo de la petición)

```
{
  "model": "gemini-3.5-flash",
  "system_instruction": {
    "parts": [{ "text":
      "Eres el community manager de Festín, café-bistró
      en La Molina. Usa SOLO la información del CONTEXTO.
      Si un dato no está, no lo inventes. Respeta el tono
      de marca y las reglas indicadas en el contexto.
      Entrega solo el texto del post, sin explicaciones.
      CONTEXTO: {{3.text}}"
    }]
  },
  "contents": [{
    "role": "user",
    "parts": [{ "type": "text", "text":
      "Escribe un post para {{1.Canal}} sobre:
      {{1.Idea Semilla}}. Comentario del revisor
      (si hay, corrígelo): {{1.Comentario Revisión}}"
    }]
  }]
}
```

La separación importa: las reglas permanentes van en system_instruction y la tarea concreta en el mensaje de usuario. {{3.text}} es el bloque agregado por el módulo 3 con los registros activos de la base de conocimiento, en formato Tema: Contenido separados por salto de línea.

3 · Gemini → Make (respuesta)

```
{
  "result": "¿Buscando un cafecito por la mañana? ...",
  "candidates": [ ... ],
  "usageMetadata": {
    "promptTokenCount": 1184,
    "candidatesTokenCount": 342,
    "totalTokenCount": 1526
  },
  "modelVersion": "gemini-3.5-flash"
}
```

Cuando la llamada falla, result vuelve como {} — y ese es justamente el valor que intercepta el filtro para no escribir contenido vacío en la base.

4 · Make → Airtable (escritura en Logs)

```
{
  "Evento": "Fallo en generación con Gemini",
  "Pieza": ["recT79iu7ql9s6z9G"],
  "Escenario": "Esc 1 - Generación",
  "Resultado": "Error",
  "Módulo": "IA - Redacción Gemini",
  "Detalle": "[404] models/gemini-modelo-inexistente"
}
```

El campo Pieza viaja como arreglo de IDs: así es como Airtable representa un vínculo entre tablas.

Criterio 3 · Optimización de costos

La consigna advierte que esto es un entregable en sí mismo y no basta con limitar tokens. Lo que sigue es la matriz de decisión que se usó para elegir modelo por tarea, el cálculo del ahorro y —porque el resultado no fue el esperado— la justificación de por qué se mantuvo Gemini pese a no ser la opción más barata.

Precios de referencia

Tarifas por millón de tokens, consultadas el 26 de septiembre de 2026 en la documentación oficial de cada proveedor.

Modelo	Entrada	Salida	Perfil de uso
Gemini 3.5 Flash-Lite	\$0,30	\$2,50	Clasificación, etiquetado, respuestas cortas
GPT-4.1 Mini	\$0,40	\$1,60	Tareas simples de alto volumen
Gemini 3.8 Flash	\$0,75	\$3,75	Redacción general (tarifa vigente hasta dic-2026)
Claude Haiku 4.5	\$1,00	\$5,00	Velocidad con buen seguimiento de instrucciones
Gemini 3.5 Flash	\$1,50	\$9,00	El modelo en producción en este sistema
GPT-4.1	\$2,00	\$8,00	Razonamiento de propósito general
Claude Sonnet 5	\$2,00	\$10,00	Lectura densa y documentos largos
Claude Opus 5.5	\$4,00	\$20,00	Razonamiento complejo, criterio fino

Fuentes: ai.google.dev/gemini-api/docs/pricing · platform.claude.com/docs/about-claude/pricing

Matriz de decisión por tarea

Tarea del ecosistema	Modelo elegido	Por qué ese y no otro
Redactar un post de 300–400 caracteres con contexto de marca	Gemini 3.5 Flash	Volumen alto, salida corta, criterio bajo. Un modelo de razonamiento aquí es gasto puro.
Clasificar una idea semilla por tipo de contenido	Flash-Lite	Decisión cerrada entre pocas opciones: no necesita capacidad de redacción.
Leer un PDF de carta nueva y poblar la base de conocimiento	Claude Sonnet 5	Lectura densa sobre documento largo, donde equivocarse contamina todo lo que venga después.
Regenerar el catálogo completo tras un cambio de carta	Batch API (–50%)	Proceso masivo sin urgencia: la mitad de precio a cambio de esperar.
Auditar 200 piezas publicadas contra el tono de marca	Haiku 4.5 + Batch	Volumen alto y juicio simple; el descuento por lote lo vuelve marginal.

Cuánto cuesta realmente

Escenario de cálculo: 1.000 piezas generadas. Cada una consume aproximadamente 1.200 tokens de entrada (contexto de la base de conocimiento más el prompt) y 350 de salida — cifras tomadas de usageMetadata en ejecuciones reales.

Modelo	Costo / 1.000 piezas	Frente al elegido
Gemini 3.5 Flash-Lite	US\$ 1,23	−3,71 más barato
GPT-4.1 Mini	US\$ 1,04	−3,91 más barato
Gemini 3.8 Flash	US\$ 2,21	−2,74 más barato
Claude Haiku 4.5	US\$ 2,95	−2,00 más barato
Gemini 3.5 Flash ← en uso	US\$ 4,95	—
GPT-4.1	US\$ 5,20	+0,25 más caro
Claude Sonnet 5	US\$ 5,90	+0,95 más caro
Claude Opus 5.5	US\$ 11,80	+6,85 más caro

El hallazgo incómodo

El modelo en producción no es el más barato de la tabla: GPT-4.1 Mini cuesta menos de una cuarta parte. Se mantuvo Gemini por tres razones que el precio por token no captura. La primera es que el nivel gratuito de Google absorbe por completo el volumen de este piloto, así que el costo real hoy es cero. La segunda es que Make tiene módulo nativo de Gemini: usar OpenAI aquí implicaría llamadas HTTP manuales, más módulos y más operaciones consumidas —el recurso que sí escasea en el plan gratuito. La tercera es la latencia: Flash responde en torno a dos segundos, y con el trigger corriendo cada quince minutos eso mantiene el ciclo holgado. Si el volumen creciera hasta salir del nivel gratuito, la migración a GPT-4.1 Mini o a Flash-Lite sería el primer ajuste a hacer.

Ahorros que no vienen del modelo

Tres decisiones de arquitectura reducen el gasto más que cualquier cambio de proveedor:

- Agregar antes de llamar. El módulo Text aggregator une los diez registros de la base de conocimiento en un solo bloque, de modo que cada pieza cuesta una llamada a la IA. Sin él harían falta diez: una por registro. Es un 90% menos de llamadas.
- Filtrar antes de gastar. El trigger descarta las filas sin idea semilla mediante `{Idea semilla}!= ""`. Una fila vacía nunca llega al modelo, así que nunca cuesta nada.
- Techo por ejecución. El límite de 10 registros impide que un error de carga masiva dispare cientos de llamadas en una sola corrida.

Consumo de operaciones en Make

En el plan gratuito el recurso escaso no es el token sino la operación: 1.000 al mes. Una pieza que recorre el ciclo completo consume seis operaciones en el primer escenario y cuatro en el segundo. Diez por pieza publicada significa un techo aproximado de 100 piezas mensuales. Si el volumen se acercara a ese límite, el ajuste correcto es espaciar el trigger de quince minutos a una hora: las corridas en vacío también consumen operaciones.

Criterio 4 · Seguridad y resiliencia

Tres frentes: qué datos se exponen, qué pasa cuando algo falla y dónde se detiene el sistema a pedir permiso.

Minimización de datos

El sistema no maneja datos personales de clientes. Lo que viaja a la API de Gemini es información comercial que el negocio ya publica en su propio perfil: carta, precios, horario, dirección y teléfono de contacto. No hay pedidos, reservas ni identidades involucradas.

Medida	Implementación
Credenciales fuera del flujo	Las conexiones de Airtable, Gemini y Gmail viven en el almacén de Make. Ninguna clave aparece en un módulo, en un prompt ni en el blueprint exportado.
Superficie mínima en el prompt	Solo entran los registros con «Activo» marcado. Apagar una casilla retira ese dato del contexto sin tocar la automatización.
Sin PII	No se procesan nombres, correos ni teléfonos de clientes.
Destinatario único	La notificación de Gmail va a la casilla del editor, no a listas externas.
Lectura sin escritura	Los enlaces públicos del dashboard son de solo lectura: nadie puede alterar datos desde ahí.

Guardarraíl contra invención de datos

Un sistema que publica en nombre de un negocio real tiene un riesgo que no es técnico sino reputacional: que el modelo invente un precio, un horario o una promoción que no existe. La instrucción de sistema lo aborda de frente —«Usa SOLO la información del CONTEXTO. Si un dato no está, no lo inventes»— y el punto de validación humana funciona como segunda barrera: aunque el modelo se saliera del guion, nada llega al público sin que alguien lo lea antes.

Rutas de error

Los dos puntos frágiles del sistema son llamadas a servicios de terceros: la API de Gemini y la de Gmail. Ambos llevan un error handler con directiva Resume, que cumple dos funciones: deja constancia del fallo en la tabla de Logs y permite que el escenario siga procesando las piezas restantes en vez de abortar la corrida completa.

Punto de fallo	Directiva	Qué hace el sistema
API de Gemini no responde o devuelve error	Resume	Escribe el mensaje literal del error en Logs y sigue con la siguiente pieza
Contenido generado vacío tras un fallo	Filtro	Bloquea la escritura: la pieza se queda en Generando en lugar de avanzar vacía
API de Gmail rechaza el envío	Resume	Registra el fallo sin marcar la pieza como publicada
Fila sin idea semilla	Fórmula del trigger	La descarta antes de entrar al flujo

El filtro que se añadió durante las pruebas

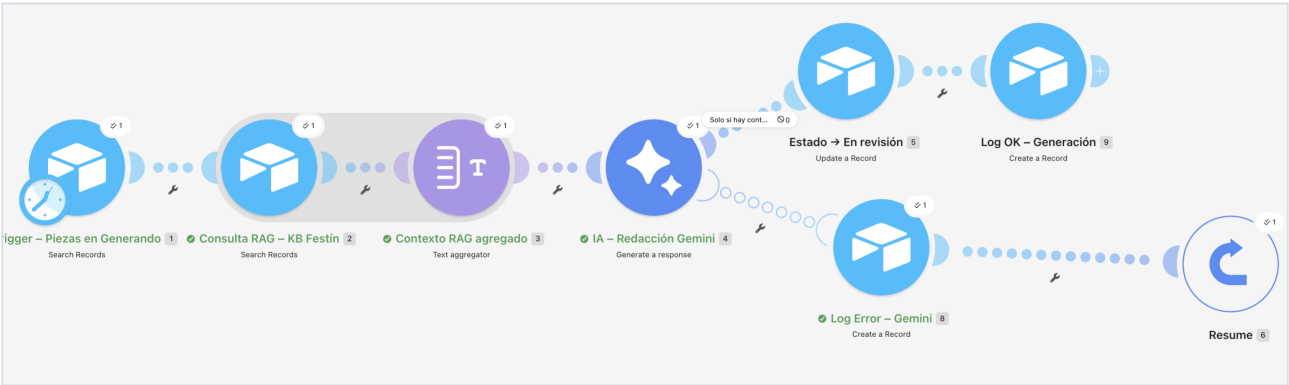
La primera versión del error handler tenía un defecto que solo apareció al probarlo. La directiva Resume, por diseño, marca el módulo como exitoso para que el flujo continúe —y eso hacía que una pieza sin contenido avanzara igual a «En revisión», registrando además un OK falso que contaminaba la tasa de error. Se corrigió insertando un filtro entre el nodo de IA y la escritura en Airtable:

```
{{4.Result}}  Text operators: Not equal to  {}
```

Cuando Gemini falla, Result vuelve como objeto vacío y el filtro corta el paso. La pieza permanece en Generando, lista para el siguiente intento, y en Logs queda solo el registro de error. Las once filas de la tabla conservan las dos versiones del comportamiento: las corridas de 12:29 y 12:36 muestran el defecto original; la de 12:38, el corregido.

Evidencia · la ruta de error funcionando

Para provocar un fallo real se apuntó el módulo de IA a un modelo inexistente (gemini-modelo-inexistente). La API devolvió 404 y el sistema reaccionó como estaba previsto.



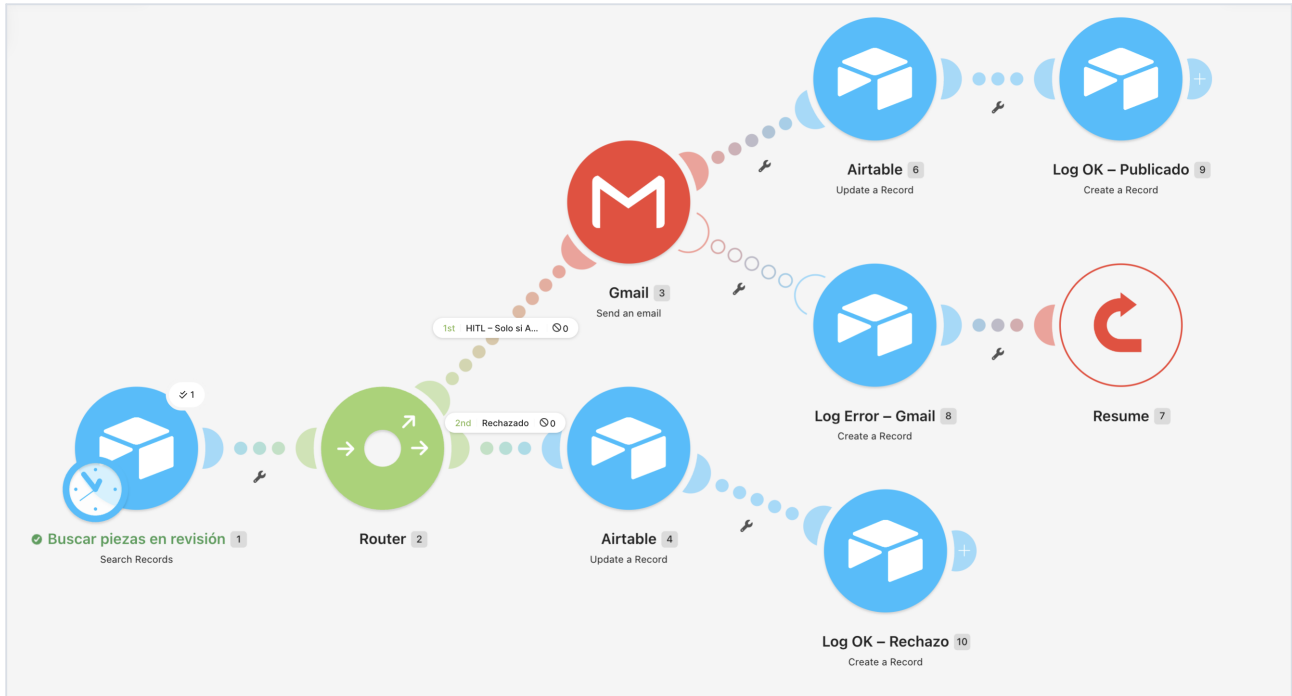
El filtro corta el paso (00). Los módulos de actualización y de log OK quedan apagados. Solo se activan Log Error y Resume. La pieza no se contamina.

	A Evento	Pieza	Escenario	Resultado	Módulo	Detalle	Timestamp	
1	Contenido generado corr...	vacía	Esc 1 - Generación	OK	IA - Redacción Gemini		26/9/2026 12:26pm	
2	Contenido generado corr...	Café de especialidad por la	Esc 1 - Generación	OK	IA - Redacción Gemini		26/9/2026 12:26pm	
3	Contenido generado corr...	Menú ejecutivo de lunes a s	Esc 1 - Generación	OK	IA - Redacción Gemini		26/9/2026 12:26pm	
4	Fallo en generación con G...	Postres artesanales del día	Esc 1 - Generación	Error	IA - Redacción Gemini	[404] models/gemini-mo...	26/9/2026 12:29pm	
5	Contenido generado corr...	Postres artesanales del día	Esc 1 - Generación	OK	IA - Redacción Gemini		26/9/2026 12:29pm	
6	Fallo en generación con G...	Postres artesanales del día	Esc 1 - Generación	Error	IA - Redacción Gemini	[404] models/gemini-mo...	26/9/2026 12:36pm	
7	Contenido generado corr...	Postres artesanales del día	Esc 1 - Generación	OK	IA - Redacción Gemini		26/9/2026 12:36pm	
8	Fallo en generación con G...	Postres artesanales del día	Esc 1 - Generación	Error	IA - Redacción Gemini	[404] models/gemini-mo...	26/9/2026 12:38pm	
+								

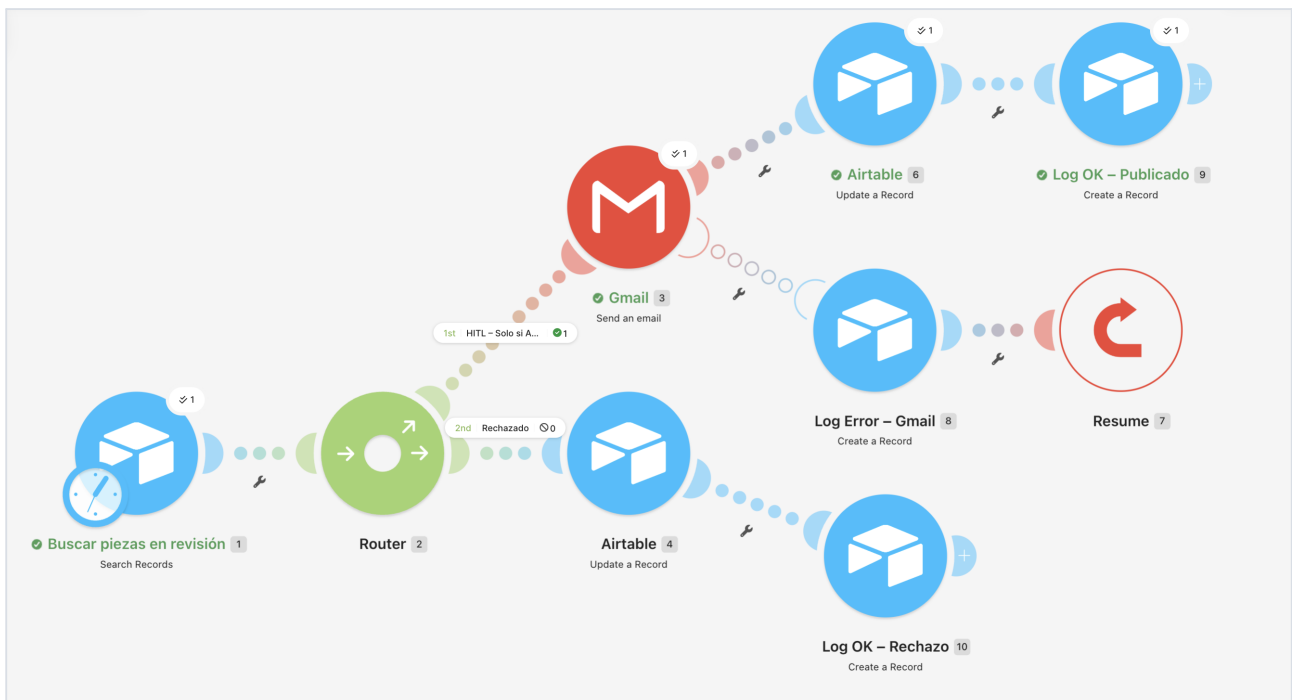
El mensaje literal de la API queda guardado en la columna Detalle: [404] models/gemini-mo...

Punto de validación humana

La consigna lo llama «efecto metralleta»: un sistema que publica sin freno. Aquí el freno es una casilla. El Router del segundo escenario evalúa si el campo Aprobado existe en el registro —y Airtable solo lo devuelve cuando alguien lo marcó—. Mientras nadie lo haga, ambas rutas quedan en cero y no ocurre absolutamente nada.



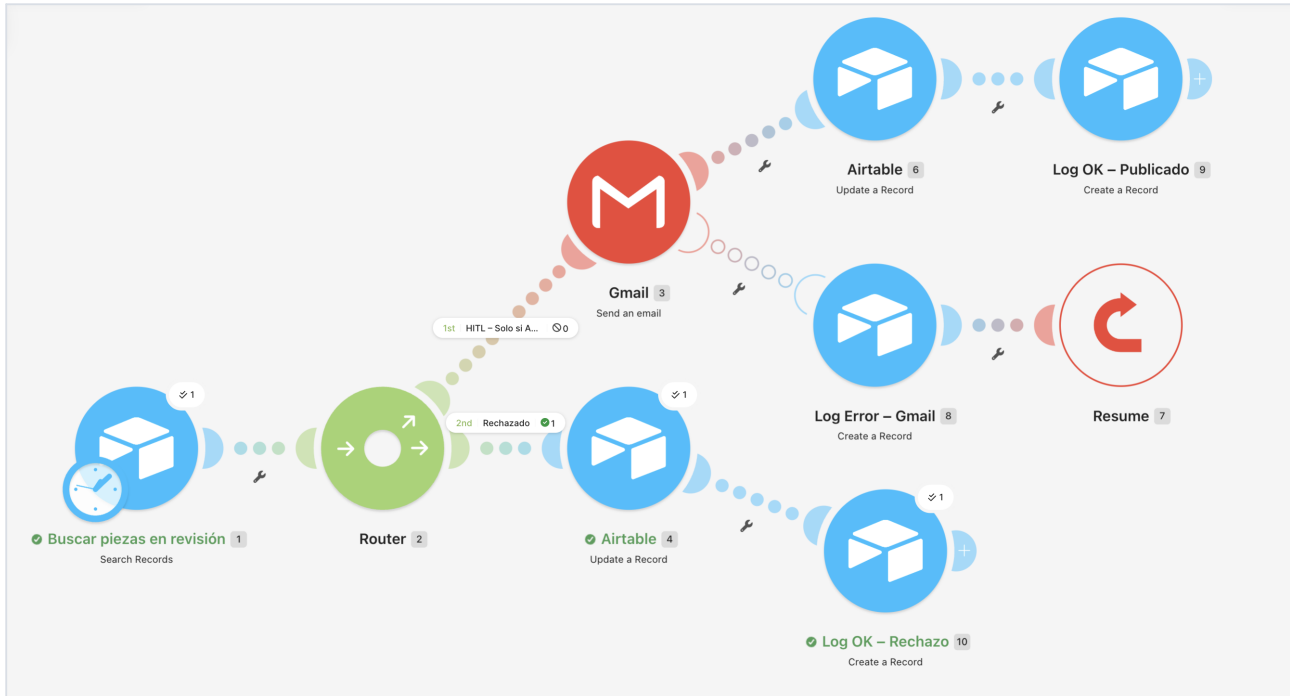
Sin aprobación. El trigger encuentra piezas, pero las dos rutas del Router muestran 0. Nada se envía, nada se publica. El sistema espera.



Con la casilla marcada. Exactamente un bundle cruza la ruta HITL: se envía el correo, la pieza pasa a Publicado y se registra el log. La ruta de rechazo permanece en cero.

La ruta de rechazo

Aprobar no es la única salida. Si el revisor escribe un comentario y marca la pieza como rechazada, el Router la manda por el segundo camino: vuelve a «Generando» conservando el comentario, que el prompt del primer escenario vuelve a leer al regenerar. El sistema no repite el mismo texto: lo corrige con la observación recibida.



Ruta 2 activa con un bundle. La pieza regresa al inicio del ciclo con el feedback incorporado.

Control anti-bucle

La pregunta del check de seguridad de la consigna —si el flujo tiene protección contra bucles infinitos— se resuelve en tres capas que actúan juntas:

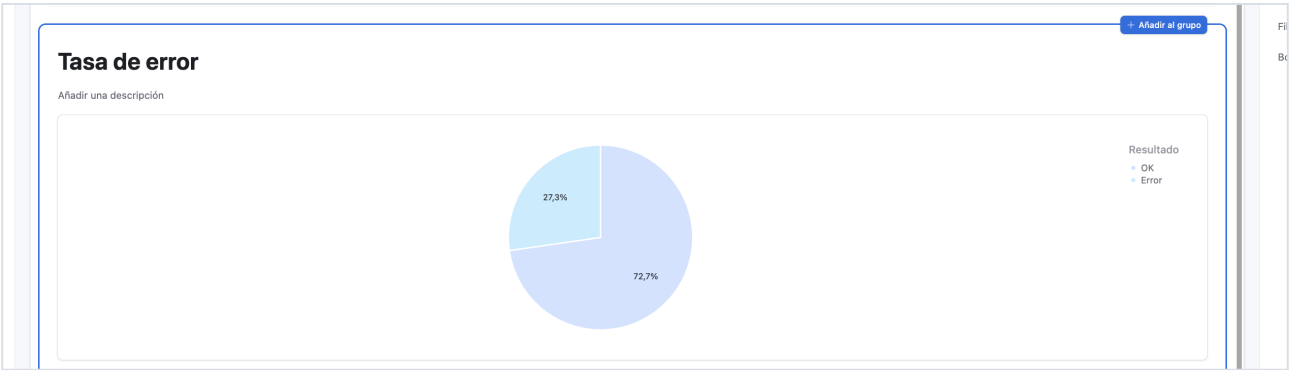
- La condición del Router excluye explícitamente Estado $\neq$ Publicado, de modo que una pieza ya publicada no puede volver a entrar.
- El trigger del segundo escenario solo busca estados «En revisión» y «Rechazado»: los estados terminales quedan fuera de su alcance.
- Ambos triggers tienen techo de 10 registros por ejecución.

Comparación de tipos en los filtros

El otro punto del check de seguridad. Durante las pruebas el Router entregaba cero bundles incluso con piezas aprobadas: la condición comparaba el campo contra el texto "Yes", mientras que Airtable devuelve un booleano. La solución no fue cambiar el texto por "true" sino eliminar la comparación de valor: se usa Exists sobre el campo. Si Airtable lo devuelve, es porque un humano lo marcó. Esa es exactamente la semántica que el control humano necesita, y además es inmune al tipo de dato.

Criterio 5 · Dashboard de control

Panel construido sobre Airtable Interfaces, alimentado en vivo por la tabla de Logs. No es un enlace a la base: es una vista de alto nivel con los indicadores del sistema.



Tasa de error en vivo: 72,7% de ejecuciones correctas frente a 27,3% con error.

Indicadores que expone

Indicador	Cómo se calcula	Valor medido
Ejecuciones totales	Recuento de filas en Logs	11
Tasa de error	Filas con Resultado = Error / total	27,3%
Tasa de éxito	Filas con Resultado = OK / total	72,7%
Carga por escenario	Agrupación por Escenario	8 generación · 3 distribución
Errores por módulo	Agrupación por Módulo	3 en IA – Redacción Gemini
Detalle del fallo	Columna Detalle de las filas con error	[404] modelo inexistente

Lectura de los números

Una tasa de error de 27,3% sería alarmante en producción, pero aquí describe una sesión de pruebas donde el fallo se provocó a propósito tres veces sobre once ejecuciones. El dato relevante no es la magnitud sino que el sistema la haya medido solo: los tres errores quedaron registrados con su causa exacta sin intervención de nadie, que es precisamente lo que un panel de control debe permitir.

Sobre el acceso público

Airtable reserva la publicación de Interfaces al plan Team. Para cumplir el requisito de enlace público se habilitaron vistas compartidas de las tablas, que ofrecen acceso de lectura sin necesidad de cuenta y muestran los mismos conteos agrupados. Los enlaces están en el anexo.

Test de estrés

La consigna pide al menos cinco ejecuciones incluyendo el «camino infeliz». Se ejecutaron seis pruebas el 26 de septiembre de 2026, tres de ellas diseñadas para romper el sistema.

#	Prueba	Qué se esperaba	Qué ocurrió
1	Generación normal con tres piezas válidas	Contenido redactado y estado movido a En revisión	Tres piezas generadas y tres registros OK en Logs
2	Pieza con idea semilla vacía	El trigger debía descartarla antes de llamar al modelo	No entró al flujo ni generó ningún log
3	Modelo de IA inexistente	Error handler activo y registro del fallo	Log Error con el mensaje 404 · flujo no se detuvo
4	Escenario 2 sin ninguna aprobación	Router en cero por ambas rutas	∅0 en las dos rutas · nada se publicó
5	Pieza con la casilla Aprobado marcada	Envío de correo y paso a Publicado	Un bundle por la ruta HITL · correo enviado · log OK
6	Pieza rechazada con comentario	Retorno a Generando conservando el feedback	Ruta 2 activa · pieza devuelta · log OK

Registro completo de ejecuciones

	A Evento	Pieza	Escenario	Resultado	A Módulo	Detalle	Timestamp	
	RESULTADO OK 8							
	1	Contenido generado corr...	vacía	Esc 1 - Generación	OK	IA - Redacción Gemini	26/9/2026 12:26pm	
	2	Contenido generado corr...	Café de especialidad por la	Esc 1 - Generación	OK	IA - Redacción Gemini	26/9/2026 12:26pm	
	3	Contenido generado corr...	Menú ejecutivo de lunes a v	Esc 1 - Generación	OK	IA - Redacción Gemini	26/9/2026 12:26pm	
	4	Contenido generado corr...	Postres artesanales del día	Esc 1 - Generación	OK	IA - Redacción Gemini	26/9/2026 12:29pm	
	5	Contenido generado corr...	Postres artesanales del día	Esc 1 - Generación	OK	IA - Redacción Gemini	26/9/2026 12:36pm	
	6	Contenido generado corr...	Postres artesanales del día	Esc 1 - Generación	OK	IA - Redacción Gemini	26/9/2026 12:51pm	
	7	Pieza publicada	Café de especialidad por la	Esc 2 - Distribución	OK	Publicación aprobada	26/9/2026 1:05pm	
	8	Pieza devuelta a corrección	Menú ejecutivo de lunes a v	Esc 2 - Distribución	OK	Ruta rechazo	26/9/2026 1:07pm	
	+							
	RESULTADO Error 3							
	9	Fallo en generación con G...	Postres artesanales del día	Esc 1 - Generación	Error	IA - Redacción Gemini	[404] models/gemini-mo... 26/9/2026 12:29pm	
	10	Fallo en generación con G...	Postres artesanales del día	Esc 1 - Generación	Error	IA - Redacción Gemini	[404] models/gemini-mo... 26/9/2026 12:36pm	
	11	Fallo en generación con G...	Postres artesanales del día	Esc 1 - Generación	Error	IA - Redacción Gemini	[404] models/gemini-mo... 26/9/2026 12:38pm	
	+							

Tabla de Logs agrupada por Resultado: 8 ejecuciones correctas y 3 con error, cada una con su módulo, su pieza vinculada y su marca temporal.

Lo que las pruebas cambiaron en el sistema

El test de estrés no confirmó que todo funcionaba: encontró dos defectos reales que obligaron a modificar la implementación. El primero fue el OK falso que acompañaba a cada error, resuelto con el filtro sobre el resultado del modelo. El segundo fue la condición del Router comparando un booleano contra una cadena de texto, resuelto pasando a una comprobación de existencia. Ambos aparecen documentados en el criterio 4, y las filas de Logs conservan el rastro de las dos versiones.

Anexo · Enlaces y archivos técnicos

Escenarios en Make (acceso público)

Recurso	Enlace
Escenario 1 — Generación de contenido	us2.make.com/public/shared-scenario/pTeeVWUStNB/festin-1-generacion-de-contenido
Escenario 2 — Distribución con HITL	us2.make.com/public/shared-scenario/4jeHK0k5v8s/festin-2-distribucion-con-hitl

Base de datos y panel de control

Recurso	Enlace
Dashboard de Control — Interfaces	airtable.com/app1KnLvAzHHtM5fP/pag6blqvunOYUbrP6
Vista pública — tabla Logs	airtable.com/app1KnLvAzHHtM5fP/shrpOjQjwl5PjRIYN/tblEs5H4KXetAG5Jv/viwC6WdmO4mdYeIUv
Vista pública — tabla Piezas	airtable.com/app1KnLvAzHHtM5fP/shrpOjQjwl5PjRIYN/tbl1aODnCVIOpLq0v/viwsKPOypw3AL44nF

Archivos incluidos en el repositorio

- blueprint-escenario-1.json — lógica completa del flujo de generación
- blueprint-escenario-2.json — lógica completa del flujo de distribución
- /evidencias/ — capturas de las seis pruebas y del panel de control
- README.md — descripción del sistema y guía de replicación

Inventario de evidencias

Evidencia	Qué demuestra	Pág.
Diagrama de arquitectura	Criterio 1 · triggers, router, APIs, IA y destino de datos	3
Canvas de ambos escenarios	Criterio 1 · nodos nombrados y rutas de error	5
Esquema relacional y contratos JSON	Criterio 2 · estructuras documentadas	6–9
Matriz de modelos y cálculo de costo	Criterio 3 · justificación por tarea	10–11
Ruta de error con fallo real de API	Criterio 4 · resiliencia	14
Router en cero sin aprobación	Criterio 4 · punto de control humano	15
Router con un bundle tras aprobar	Criterio 4 · publicación condicionada	15
Ruta de rechazo activa	Criterio 4 · ciclo de corrección	16
Panel de tasa de error	Criterio 5 · dashboard en vivo	17
Tabla de Logs agrupada	Test de estrés · once ejecuciones	18